

Linux系统与安全

▼ 第一章 Linux系统与安全

▼ 系统漏洞主要包括

- 1.系统输入验证漏洞
- 2.缓冲区溢出漏洞
- 3.边界条件系统漏洞
- 4.访问校验系统漏洞
- 5.意外逻辑漏洞
- 6.竞争条件漏洞
- 7.系统顺序化操作漏洞
- 8.系统环境漏洞
- 9.系统配置漏洞

▼ 操作系统安全管理的主要目标

- 1.依据系统安全策略对用户的操作进行**访问控制**，防止用户对信息资源的非法访问（窃取、篡改和破坏）
- 2.标识系统中的用户和进行**身份鉴别**
- 3.监督系统运行时的**安全性**
- 4.保证**系统自身**的安全性和完整性
- 5.建立**访问控制**机制：自主访问控制、安全标记与强制访问控制、客体重用等
- 6.建立**可追究**机制：标识与鉴别、可信路径、安全审计等
- 7.建立**持续保护**机制：系统完整性、隐蔽通道分析、最小特权管理、可信恢复等

▼ Linux系统简介

荷兰计算机科学家Andy Tanenbaum开发了 类Unix操作系统-Minix，主要应用于教学实验。

芬兰大学生Linus Torvalds在Minix基础上编写了一个操作系统核心软件，并于**1991年10月公布为Linux 0.0.2版**。

很多商业公司可以**免费得到Linux的核心源代码**，并加以包装和改进后形成自己的产品， 即Linux发行套件(distribution)。

▼ Linux操作系统的安全问题

清理系统不需要的用户和用户组

关闭系统不需要的服务

禁止非root用户执行/etc/rc.d/init.d/下的系统命令

限制文的权限

禁止使用CTRL+Alt+Del快捷键重启服务器

关于系统更新的安全配置的安全

/tmp、/var/tmp安全设定

修改history命令记录

隐藏服务器系统信息

▼ 第二章 系统入门

默认情况下超级用户的提示符是：#

默认情况下普通用户的C-shell提示符是：%

默认情况下普通用户的b-shell提示符是：\$

字符界面下，使用init命令关机所用的参数是：0

字符界面下，使用init命令重启系统所用的参数是：6

使用shutdown命令关机的参数是：-h

使用shutdown命令重启的参数是：-r

i节点：linux中，文件查找不是通过文件名称来查找的。实际上是通过i节点来实现文件的查找定位的。我们可以形象的将i节点看做是一个指针fip。

Linux系统的三种基本文件类型：普通文件、目录文件、设备文件

▼ 简答题

▼ Linux系统下字符界面的优势是什么？主要提示符是什么？

优势一：字符界面占用的系统资源更少，更加适合服务器

优势二：字符界面减少了出错、被攻击的可能性

主要提示符：login

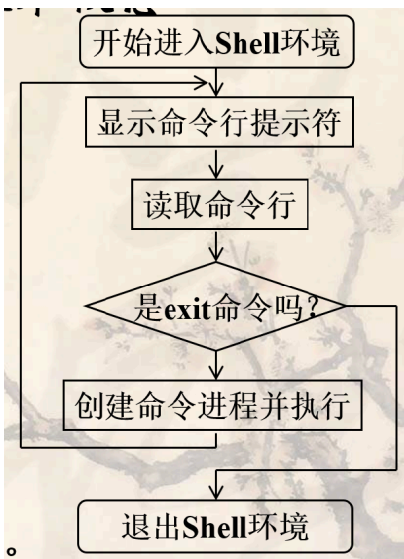
▼ Linux如何进行本地登录和注销？

登录：在login提示符下，输入用户名，然后回车，在passwd提示下，正确输入密码进行登录。

注销：在操作系统shell提示符（\$或者%）下，输入logout命令，或者^d命令等

▼ Linux系统的shell的主要六个功能以及工作流程？

命令解释执行；文件名及管理；I/O重定向；通信管道建立；系统环境设置；shell编程



▼ Linux系统的通配符的含义？

*：代表从它所在位置开始的任意字符串，但，文件名中的“.”和路径名中“/”必须显式匹配。

?：代表它所在位置上的任何单个字符

[]：每次只匹配“[]”中字符的一个。方括号中的字符范围可以由直接给出的字符组成，也可以由表示限定范围的起始字符、终止字符及中间的连字符“-”组成

\：转义字符，用于对某些特殊字符的表示成本身的含义，略去其特殊含义

▼ 一些快捷键

Ctrl+D 结束当前程序输入或结束当前程序，或从系统中注销

Ctrl+C 终止当前程序的执行

Ctrl+Alt+F# 字符终端切换，如Ctrl+Alt+F2将终端切换至F2

▼ 目录

. —当前目录；.. —上级目录或者是父目录；

根目录是指文件系统创建时自动生成的唯一顶级目录，用/来表示

▼ 设备文件及其特点

UNIX/Linux把每一个I/O设备都看成一个文件，与普通文件一样处理，这样可以使文件与设备的操作尽可能统一。从用户角度来看，对I/O设备的使用和对一般文件的使用一样，不必了解I/O设备的细节。

▼ Linux开关机命令

halt：关闭系统 用法halt [-f] [-p]

-f用于强制快速关机或重启；参数-p用于在关闭系统时关掉电源

reboot：与halt几乎一样，但其在关闭系统时会重新启动它

poweroff：等价于halt -p

shutdown：在关机前向系统内的所有用户发送通知或警告信息 用法：shutdown [-krhfFc] time [wall msg]

-r	重启计算机
-h/-H	关机后关闭电源
-k	并不真正关机，只是发送警告信号给每位用户
-c	取消目前已经提交的shutdown任务。该选项同时使用可有一个用来解释撤销关机的广播信息
time	指定关机时间。可为格式为hh:mm的绝对时间，如1:20，或相对时间（单位为分），如+10表示10分钟后关机，+0表示now。若不指定则默认为+1
wall msg	广播信息。若不指定，系统提供默认信息

▼ 第三章 Shell与Shell基本操作

*：代表从它所在位置开始的任意字符串，但文件名中的“.”和路径名中“/”必须显式匹配。

?：代表它所在位置上的任何单个字符。[]：每次只匹配“[]”中字符的一个。

特殊字符“\”被称为转义字符（ESCape char），用于对某些特殊字符的表示。符号“\\”表示“\”

Ctrl+D：结束当前程序输入或结束当前程序，或从系统中注销

Alt+F#：字符终端切换，如Alt+F2将终端切换至F2

在UNIX/Linux系统中，目录只包括文件名和i节点号等相关信息，而文件的属性信息保存在i节点信息中。

目录文件在形式上同普通文件一样，但具有目录属性，只能用目录管理命令来访问和管理。

UNIX/Linux把每一个I/O设备都看成一个文件，与普通文件一样处理。

▼ 一些命令

\$ ls -l /dev | wc -l：以长列表的形式列出目录/dev下的内容，并直接对显示文件个数

\$ x=123：在\$系统提示符下，定义一变量x，值为123

\$ echo \$PATH：在shell提示符下，显示当前定义的命令搜索路径

▼ 转义字符

特殊字符“\”被称为转义字符（ESCAPE char），用于对某些特殊字符的表示。Shell中“\”的作用与在C语言中的作用类似。“\”转义的特殊字符

字 符	意 义	字 符	意 义	字 符	意 义
\a	响铃符	\x??	16进制表示	\\	\
\b	退格键	\0???	8进制表示	\`	`
\f	换页	\t	Tab键	\”	”
\n	换行	\r	回车	\e	ESC

▼ 简答题

▼ 什么是无名管道？什么是命名管道？

无名管道：主要用于有直接继承关系的父子进程之间的通信。当创建无名管道的进程结束后，无名管道自动消失。

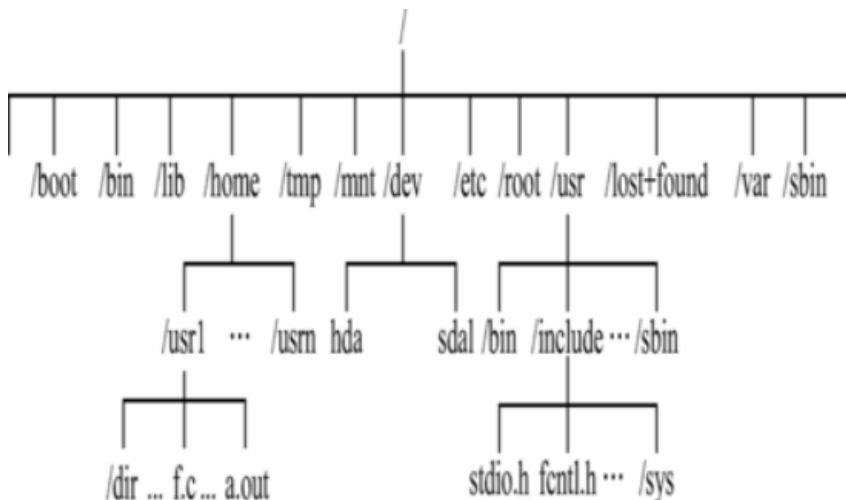
命名管道：是一种特殊的设备文件，在文件系统里以文件形式的存在。由于是以文件形式存在的，使得不论是否有家族关系的进程间都可使用其进行通信。在通信结束后，若不主动删除它，它不会自动消失。

▼ 什么是硬链接？什么是软连接（符号链接）？

硬链接（-）：两个或多个文件名共用一个文件体，也就是说一个文件可以具有多个不同的名字，但它们具有相同的i节点号。

符号链接（l）：与硬链接不同，它们是不同的文件，各有自己的i节点，但可以通过符号链接文件的内容访问被链接文件。符号链接也叫软链接。

▼ Linux系统的树目录结构



▼ 什么是绝对路径？什么是相对路径？

文件在文件系统中的位置可以由相对路径或绝对路径来决定。

绝对路径是指从“根”开始的路径；相对路径是从用户工作目录开始的路径。

绝对路径是确定不变的，而相对路径则随着用户工作目录的变化而不断变化。

▼ 重定向

标准输入重定向：是指让命令（程序）的标准输入从指定的文件中读取。因此，输入重定向主要用于解决命令的输入源数据，尤其是输入数据量大或重复使用问题。

标准输出重定向：是指把命令的标准输出保存到文件中，可用两种方式：覆盖方式(>)和追加方式(>>) 标准输出的重定向是有副作用的：一是它可能覆盖已经存在的文件；二是它可能会在一个具有完整意义的文件后面追加一些外来或无意义的信息。

标准错误的重定向：标准错误的重定向与标准输出的重定向意义相同，但是针对标准错误的，但在对其改道时须使用文件号2。命令 2> 文件 或命令 2>> 文件 在2与>间、>与>>间不能有空格

标准输出、错误同时重定向：使用符号&>或&>>可以让一个程序的标准输出和标准错误同时重定向到文件，前者用于覆盖方式，后者用于追加方式。

▼ 什么是环境变量

shell在开始执行时就已经定义了一些和系统工作环境有关的变量，用来控制用户程序的执行，或为用户程序的执行提供环境支持。

可用env、export或set来查询环境变量：

\$ export #显示所有环境变量

\$ env #同上

▼ 常用的环境变量

HOME：家目录。

LOGNAME、USER：登录用户名、用户名

IFS：命令行参数、选项间的分隔符。

PATH：由冒号分隔的目录路径名（命令执行路径）。

TERM：终端的类型。

PWD：当前工作目录的绝对路径名。

OLDPWD：刚刚离开的目录。

PS1：主提示符。

PS2：辅提示符

▼ 反撇号（反单引号）的作用

反单引号的作用是命令替换。命令替换也叫命令扩展。

所谓命令替换是指反单引号内的内容将作为命令首先被执行，然后再将命令执行的标准输出替换反单引号及其括号位置的信息。例如：

\$ x=`pwd` #通过命令替换定义变量x

\$ y=`whoami` #通过命令替换定义变量y

\$ echo \$x \$y #显示变量x和y的值

▼ 什么是内部命令？什么是外部命令？

内部命令：shell拥有自己的内部命令，也能被系统中其他应用程序所调用。比如cd；

外部命令：是存在于文件系统中某个目录下的（系统或用户）程序；

▼ 命令的搜索方式是什么？

默认情况下，当用户从键盘输入一个命令时，shell首先检查它是否带有路径。若有路径则按路径搜索它，若找到则执行之，否则报错。

若没有带路径，则首先检查它是否为内部命令，若是则执行之；否则，再沿\$PATH指定的路径检查是否是一个外部命令。如果能够成功找到，则执行之；否则报错。

▼ 基本目录操作命令

1. 列目录内容 (ls)
2. 建立目录 (mkdir)
3. 删除目录 (rmdir)
4. 改变工作目录 (cd)
5. 显示当前目录 (pwd)

▼ ls命令结果中各域名的意义

#ls -ltr s* #列出当前目录下所有名字以s开头的文件，按修改时间倒排序

#ls -lR /bin #递归列/bin目录及其子目录

#ls -ad .* #列所有名以点开头的文件

#ls -AF #列当前目录，名后属性字符

#ls -lid /usr/bin/ls /dev/sda /etc #以长格式列文件且输出I节点号

4) ls的输出与文件属性

◆ ls -lid /usr/bin/ls /dev/sda /etc的有以下内容：

◆ 2039 b rw-rw----. 1 root disk 8,0 Mar 8 09:06 /dev/sda

◆ 651522 d rwxr-xr-x. 141 root root 2288 Mar 8 09:08 /etc

◆ 894 - rwxr-xr-x. 1 root root 14159 Nov 1 01:21 /usr/bin/ls

◆ ① ② ③ ④ ⑤ ⑥ ⑦ ⑧ ⑨

各域的意义是：①I节点号；②文件类型；③权限；④链接数；⑤主；⑥组；⑦普通文件为长度，特别文件为主次设备号；⑧最后修改时间；⑨文件名。

▼ 文件操作基本命令

1. 显示文件的内容或合并文件 (cat)
2. 分屏显示文件内容 (more) 分页命令家族中的一员

less命令和more一样都是分页处理命令。它改进了more命令中的一些功能，并又添加了许多其他特色，例如，可以使用PgUp和PgDn翻页，使用HOME和END直接跳到文件开始或结尾等。

3. 使用less命令浏览文件
4. 文件或目录的删除 (rm, unlink)
5. 文件移动或更名 (mv)
6. 文件和目录复制 (cp)

▼ 反单引号的作用是命令替换。命令替换也叫命令扩展。写出命令序列①\$ x=`pwd` ②\$echo \$x的功能

将当前工作目录赋予变量x，然后显示变量x的值;显示当前工作目录

▼ vi

▼ 三种工作模式及其解释

- (1) 命令模式：当进入vi时，处在命令模式。在此模式下，可通过vi命令对文件内容进行处理，也可通过插入命令进入编辑模式
- (2) 编辑模式：可在命令模式下通过o, O, a, A, i, I等命令使vi进入编辑模式。也可通过按Esc键返回命令模式。

(3) 命令项模式或叫底行模式：在命令模式下，用户输入冒号“:”后，光标会跳到底行，输入相关命令后可完成指定操作。

(4) vi [options] [files]

▼ vi的基本操作

- (1) 编辑模式
- (2) 删除命令
- (3) 修改或替换
- (4) 搜索与定位
- (5) 搜索和替换
- (6) 块操作命令
- (7) 缓冲区操作命令
- (8) 其他命令
- (9) 保存与退出

▼ 进程和信号

进程间可以通过**信号**进行**通信**，以实现进程间一定程序的控制，用于进程间信号通信的命令是**kill**和**trap**

信号是linux系统为了**响应某些状况而产生的事件**。进程收到信号后应该采取相应的**动作**

▼ kill -9 3231 命令的功能

发送信号9（KILL）终止pid为3231的进程

▼ pstree -p| more 命令的功能是什么？

分屏显示系统进程树

▼ 别名管理

alias：别名定义

alias #显示所有已经定义的别名

unalias：别名取消

#unalias ll #取消别名ll #unalias -a #取消所有别名

▼ 第四章 用户、组和密码及安全管理

▼ 选择题

1、在UNIX/Linux系统添加新用户的命令是（）

A、 groupadd

B、 usermod

C、 userdel

D、 useradd

2、添加用户时使用参数（）可以指定用户标识号

A、 -d

B、 -p

C、 -u

D、 -c

3、用户mytest修改自己的密码可使用（ ）

A、 *passwd*

B、 *passwd -d mytest*

C、 *passwd mytest*

D、 *passwd -l*

4、解锁一个用户的命令和选项为（ ）

A、 *passwd -u*

B、 *passwd -x*

C、 *passwd -d*

D、 *passwd -l*

5、删除一个用户密码所使用的命令和选项（ ）

A、 *passwd -u*

B、 *passwd -x*

C、 *passwd -d*

D、 *passwd -l*

▼ 简答题

▼ 为了提高系统的安全性，对用户密码应如何管理？

UNIX/Linux系统的标准安全级别为C2级(TCSEC标准)，具有用户身份认证、访问控制和操作的可靠性等特点。密码管理是实现身份认证的基础。

可用于密码的字符是广泛的，视不同系统而定。一般来说，可以使用大小写字母、数字、标点符号等，有的系统甚至可以使用光标键和功能键等。密码的长度也视具体系统或安全级而定。

系统的安全与方便是矛盾的，不能因方便而忽略安全问题，也不能因过于强调安全而影响正常使用，应该在安全和方便之间找到平衡点或进行取舍。

- (1) 密码应该按规定定期或不定期修改。
- (2) 密码内不应包含完整的单词、生日、电话号码、姓名、用户名、组名、宠物和地址等信息。
- (3) 不同系统和用户应该有不同密码。
- (4) 密码应易记且不要写在纸上。
- (5) 要保密，不要共用密码。
- (6) 输入密码时不要让别人看见，也不要窥视他人密码。

▼ 简述 /etc/passwd、/etc/shadow、/etc/group文件的作用及其简要结构

▼ /etc/passwd是系统用户数据库文件，它包括系统内所有已经注册用户的信息。

该文件是一个文本文件，它的每一行描述一个用户的信息，为由“:”分隔的7个字段。

结构：username:[password]:uid:gid:[comment]:dir:[shell]

username：用户名，必须以字符开始，只能用小写字母。

password：密码。

uid：用户标识号。取值范围在/etc/login.defs文件中定义。一般用户uid满足：UID_MIN≤uid≤UID_MAX。

gid：组标识号。说明同uid。

comment：说明域。可以为空，但不能包含冒号

dir：用户家目录，即用户登录成功后所在的工作目录。

shell：用户登录成功后所使用的shell程序

▼ /etc/shadow是影子密码文件

当系统启用影子密码时，用于存放系统内用户加密后的密码和用户登录控制信息（增加安全性的一个技巧）。

结构：username:password:lastchanged:min:max:warn:inactive:expired:resv

用户一般情况下不需要直接操作/etc/shadow文件。

username：/etc/passwd内对应的用户名。

password：加密后的密码。若为空表示无密码，开头为“!”时表示用户被上锁。

lastchanged：最后修改日期。从19700101到上次修改密码的天数间隔。

min：必须修改密码的剩余天数。

max：密码的最长有效天数。warn：更换前警告的天数。

inactive：账号被取消激活前的天数。

expired：账号不活跃的天数。resv：保留

▼ /etc/group是组定义文件。它是一个文本文件，每行描述一个组。

结构：groupname:password:gid:userlist

groupname：组名。

password：组密码。可用gpasswd设置此密码，并由newgrp等命令使用。

gid：组标识号。

userlist：组成员，为以逗号分隔的用户名。

▼ 为何要上锁一个用户？如何锁定一个用户？当需要时如何解锁？

一个用户可被上锁，上锁后的用户一经退出将不能再登录；只有root能够上锁或解锁一个用户，一般情况下当不允许一个用户登录系统时，可以对其进行上锁。

```
#passwd -l user #对用户user上锁
```

```
#usermod -L user #同上
```

```
#passwd -u user #对用户user解锁
```

```
#usermod -U user #同上
```

▼ 如何在删除一个用户时也同时删除它的家目录？在操作时应该注意什么？

对于系统中不再使用的用户账号，应尽早删除它，因为多余用户的存在是安全隐患。删除用户的命令是userdel。

用法为：userdel [-r] user

选项-r用于在删除用户的同时，删除用户的家目录。

删除家目录时一定要注意确保该用户的文件等信息真正不要了，或者已经做好了备份。

▼ 如何确定用户所使用的终端？

tty用于显示命令自己使用的终端设备，以确定使用位置。

用法为：tty [-s]

若不使用选项-s，则在命令所用终端上显示所用的终端名。当使用-s时，什么也不输出，只返回状态。

状态为0时，表示命令是在终端上运行的，否则表示是在后台等非终端环境下运行的。

▼ 简述Linux系统的访问控制机制

访问控制（或权限管理）是指限制对文件、目录和其他对象的读取、写入和执行操作的能力。通过适当的设置访问控制规则，用户可以避免未经授权的操作并对特定资源进行完全掌控。

▼ 简述su和sudo命令的用法

su的功能是在使用者不退出系统的情况下以其他新用户的身份工作。当不指定任务时，将以新用户的身份启动一个shell；当指定任务时，则以新用户的身份执行任务。在以新用户工作的过程中，有效uid和有效gid变为新用户的uid和gid。

用法：su [options] [-] [newuser [args]]

sudo也允许用户以超级用户或其他用户的身份执行命令。

用法：sudo -h|-k|-l|-v|-V

▼ 简述如何进行用户和组管理

对UNIX/Linux系统的管理与使用是以用户（user）为主体的。“自然人”使用系统，必须以某个已在系统中存在的用户身份登录，且通过密码验证后才能进入系统，并按权限使用（安全性）。

系统中有两类用户：超级用户和一般用户。超级用户有至高无上的权力（责任最大，安全性要求最高），可做任何操作；一般用户只能在给定权限范围内进行工作。

用户又可分成不同的组（group）。同组用户按规定和目的享有某些共同的权限。

与用户管理相关文件：

（与Linux系统安全密切相关的文件）/etc/passwd、/etc/shadow、/etc/group、/etc/login.defs、/etc/default/useradd等，用于对用户设置和登录项目进行控制

用于用户管理的命令有useradd、usermod和userdel，它们分别用于用户创建、用户属性修改和用户删除。

▼ 用户&用户组 超级用户&一般用户

系统中有两类用户：超级用户和一般用户。超级用户有至高无上的权力（责任最大，安全性要求最高），可做任何操作；一般用户只能在给定权限范围内进行工作。

用户又可分成不同的组（group）。同组用户按规定和目的享有某些共同的权限。

▼ uid和gid

gid：group identification

一个组是具有某种关联的用户集合。比如，在某种业务方面，需要把操作某些数据文件或数据库的用户放在一个组中，以实现数据共享或共同操作。一个组中可以包含多个用户，一个用户可参与不同的组。

▼ 用户的添加与删除

▼ useradd用户创建

用法：useradd [options] user #添加用户

useradd -D #显示默认值

useradd -D [options] #设置默认值

若不带选项运行useradd，则使用默认方式创建newuser。若在创建用户时没有创建家目录，则为保证newuser创建后可以登录还必须手工创建newuser的家目录，并修改家目录归属关系和存取权限。

▼ usermod用户修改

用法为：usermod [options] [-l newname] [-L|-U] 多数选项与useradd的选项相同且意义相同

-l newname：设置用户新名字，将用户名由原来的name变为newname。

-L：将用户密码上锁 -U：将用户密码解锁。

▼ userdel用户删除

用法为：userdel [-r] user

选项-r用于在删除用户的同时，删除用户的家目录

▼ whoami

用于显示与当前有效id相关的用户名，也就是显示使用者自己的用户名，功能相当于id -uno。

▼ 第五章 文件系统安全使用及管理

UNIX/Linux用**文件存取控制表**来解决存取权限的控制问题

存取控制表以**文件**为单位，把**用户**按某种关系分为若干类，同时规定每类用户的**存取权限**

文件或目录的默认权限由文件权限**掩码umask**来控制，是权限的**补码**

默认情况，root的umask值为**0022**，普通用户的umask值为**0002**

umask命令的功能是设置或查询**umask**的值

创建文件时，系统将用八进制数**777**与文件**创建掩码**的八进制数按位做**减法**运算，所得的三位八进制数作为新文件的存取权限

若umask的值为022，则创建“文件”的理论权限为 $777-022=755$ ，即字母表示为**rwxr-xr-x**

对于某个文件，不一定具有可执行的内容，因此还要去除各类用户的执行权，故一般文件的权限为**644**，即字母表示**rw-r--r--**。

chmod（change mode）命令的功能为改变文件或目录的**访问权限**。在设置或改变文件或目录权限时，可以使用**符号或数字**两种方式。超级用户或用户主可以执行该命令。

chown（change owner）的功能是改变**文件所有者**，本质上是改变文件的**uid;UID**。只有**超级用户;root**有权使用chown

mkfs是创建文件系统的总控程序，它会根据将要创建的文件系统类型去调用相应的创建程序来完成文件系统的创建工作。

欲使用已经存在的文件系统，必须首先使用安装命令**mount**进行安装，然后才能使用，使用完毕后要用拆卸命令**umount**进行拆卸

如果不按规定操作卸载文件系统而中途把介质强行取出，则可能造成存储介质上文件系统的**损坏、数据不完整、数据丢失**

受损的文件系统必须经过**清理或修复**后才能再继续使用。使用的命令是**fsck**

在UNIX/Linux系统中，所有可以访问的文件或文件系统都必须在文件系统的目录树上的**某个目录上**。这个用于安装的位置（目录）被称为**安装点** UNIX/Linux系统提供了一个可以被广泛使用的**传统安装点/mnt**

find命令用于**文件的查找**，从而**定位**文件在指定文件系统中的位置。还可以利用它与其他命令配合作，而**完成特定任务**

ln的功能是**管理链接**，可用于创建文件的**硬链接和符号链接**

▼ 一些命令

#**chown zhang *.c**：将当前目录下所有C语言文件（扩展名为.c）的所有者改为zhang

chgrp（change group）：改变文件所属组

mkfs -t vfat /dev/sdb：在移动磁盘上创建vfat文件系统

chmod u+rw,go+rx myp：为myp的文件主增加所有权限，为组和其他人增加读和执行权

#**mount -t vfat /dev/sda1 /mnt/usb**：将U盘上vfat格式的系统安装在/mnt/usb

find /tmp /home -mtime -1 -print：在/tmp和/home下查找24小时内没有更改的文件

find /home -user gjshao -name "*.c" -print：在/home下查找属于用户gjshao的C语言程序

find /sbin /usr/sbin -executable ! -readable -print：在/sbin和/usr/sbin可执行但不可读的文件

ln -s /tmp/myf my_link：在当前目录内建立/tmp/myf的符号链接my_link

▼ UNIX/Linux的文件系统除具有一般文件系统特点外，还具有以下特点：

- (1) 文件是无结构的**字符流**。
- (2) 文件可通过**设置权限**的办法加以保护。
- (3) 外部设备都被看成**文件**，在系统内与**普通文件统一**管理。

▼ 三种普通权限及其功能

- (1) 读权限 (r)：指用户对文件或目录查看权限。
- (2) 写权限 (w)：指用户对文件或目录的写权力。若用户对某文件没有写权限，则不能修改它；若用户对某目录没有写权限，则不能在该目录内创建文件或子目录。
- (3) 执行权 (x)：指用户对文件的执行权或对目录的进入权限。若用户对某文件没有执行权则不能执行它；若用户对某目录没有执行权则不能进入它。

▼ 改变文件的权限chmod (change mode)

超级用户或用户主可以执行

- (1) 符号方式：

```
chmod [options] <a|u|g|o> <+|-|=> <str_perm>,... file ...
```

- (2) 数值方式：chmod [options] num_perm file ...

```
#chmod a+x my_p1 my_p2 #为所有用户对文件my_p1、my_p2增加执行权
```

```
#chmod u+rwx,go+rx myp #为文件主增加所有权限，为组和其他人增加读和执行权
```

```
#chmod -R o-rwx /tmp/w123 #递归去掉/tmp/w123内目录和文件对其他人的所有权限
```

```
#chmod -R 777 /tmp/* #将/tmp中的所有文件和各级子目录及内容权限设置为777
```

▼ 改变文件的所有者chown (change owner)

本质上是改变文件的uid 只有超级用户有权使用

其用法为：chown [options] owner[:group] file ...

```
chown [ options ] :group file ...
```

如果用户组被指定，则文件的所属组也将被修改

```
#chown root my_files #将my_files的所有者改为root
```

```
#chown zhang *.c #将当前目录下所有C语言文件的所有者改为zhang
```

```
#chown -R gjshao /tmp/mydir #递归将/tmp/mydir的所有者改为gjshao
```

▼ SCSI (Small Computer System Interface) 储存设备命名

与IDE硬盘设备的命名方法相同

例如，/dev/sda、/dev/sdb分别为第1、2个SCSI硬盘，/dev/sda1、/dev/sda2分别代表第1个SCSI硬盘的第1、2分区

▼ 文件挂载

欲使用已经存在的文件系统，**必须首先使用安装命令mount**进行安装，然后才能使用，使用完毕后用**拆卸命令umount**进行拆卸。

必须按“**安装**→**使用**→**拆卸**”顺序来进行操作。

如果不按规定操作而中途把**介质强行取出**，则可能**造成存储介质上文件系统的损坏、数据不完整或丢失**。

受损的文件系统必须经过（使用**fsck**）**清理或修复**后才能再继续使用。

▼ find

用法为：find [path ...] [expression]

在find的所有参数中，位于命令名后第一个选项前的参数为查找位置，若无则默认为当前目录。如果也没有指定参数或查找指标，则默认为-print

使用示例

- (1) 按修改时间查找文件。

```
#find /tmp /home -mtime -1 -print    #在/tmp和/home下查找24小时内没有更改的文件
```

```
find $HOME -mtime 0 #在/home下查找24小时内被更改的文件
```

- (2) 在/home下查找属于用户gjshao的C语言程序。

```
#find /home -user gjshao -name ".c" -print
```

- (3) 将当前目录内的所有.c和*.h的文件打成tar包/tmp/mych.tar。

```
#tar cvf /tmp/mych.tar 'find . -name "*.ch" -type f -print'
```

- (4) 在/home下查找所有权限为0644用户主为test的文件。

```
#find /home -perm 0644 -user test -print
```

▼ df和du的区别

df：按指定格式显示系统中已安装文件系统的使用情况。

用法：df [options] [filesystem]

```
#df -v    #显示文件系统的使用情况
```

```
#df -i    #显示系统i节点的使用情况
```

du：按指定单位统计文件或目录的大小。

用法：du [options] [names]

▼ 简答题

▼ UNIX/Linux系统对文件操作的三种权限只对三类人分配，是哪三类人，分别进行解释。

分别是用户主（user：u）、同组人（group：g）和其他人（other：o）。

用户主是文件的拥有者；

同组人是与文件主同组的用户；

其他人是指除用户主和同组人以外的用户。

▼ 简述权限的字符串表示方法

对某类用户的权限分配有3种：

- (1) 读权限。对应位置为r，否则为-。

- (2) 写权限。对应写权限位置为w，否则为-。

- (3) 执行权限。若用户对文件有执行权或对目录有进入权，则对应执行权限的位置为x，否则为-。

▼ 简述权限的数字表示方法

在数字方式表示下，访问权限可用一个三位二进制数来表示。

若具有某个权限，则对应位为1，否则为0，这样某类用户的权限就可表示为三位二进制数，其值为不大于7的整数，因此可作为八进制数。例如，目录/dev的权限为rwxr-xr-x，它的主、组和其他人的权限分别为rwx、r-x和r-x，表示为二进制数为111、101和101，分别转换为八进制数则为7、5和5。此时，可以说/dev的权限为755

▼ 简述umask、文件和目录的默认权限之间的对应关系

	root	普通用户
umask	u=rwx,g=rx,o=rx / 022	u=rwx,g=rwx,o=rx / 002

文件	rw-r--r-- / 644	rw-rw-r-- / 664
目录	rwxr-xr-x / 755	rwxrwxr-x / 775

▼ 什么是文件系统？使用文件系统前需要做哪些工作？

文件系统是建立在存储介质上的，对存储介质的使用要首先进行规划，如对硬盘的使用必须先在其上创建（物理或逻辑分区）分区等。

其次是对分区进行格式化，在其上创建文件系统，然后才能使用。很多时候，用户使用设备时并没有做这些工作，那是因为在这之前已经做过了。

▼ 第六章 进程及安全管理

程序是一个存储在介质上的，既具有可执行的属性，又具有执行的内容文件。

进程是一个程序的动态执行过程。

在多任务、多用户系统中进程具有并发和并行特征。在一个系统内部，可以同时运行很多进程，甚至可以让同一个程序多次同时运行，但它们对应不同的进程，在系统内有不同的进程标识，也就是说不同的进程可以执行同一个程序，但可能使该程序所处理的数据集有所不同。

作业或任务是用户需要计算机完成某项任务时要求计算机所做工作的集合，一个作业可能需要几个程序联合完成。

▼ 进程状态及查询（ps）

ps的功能是显示进程的状态和信息，给出系统当前正在运行进程信息的快照，可以配合kill结束系统中失控或不必要的进程。

用法为：ps [options]

参 数	功 能 说 明
-A, -e	显示系统内的所有进程
-a	显示所有与终端相关的进程
-l, l	以长格式显示进程信息

▼ 选择题

若使pid进程无条件终止，使用的命令是（ ）

- A、 kill -9
- B、 kill -15
- C、 killall -1
- D、 kill -3

在Linux系统中用于进程状态查询的常用命令是（ ）和（ ）

- A、 ps
- B、 kill
- C、 killall
- D、 pstree

▼ 简答题

- ▼ UNIX/Linux系统有几类进程？试说明后台进程的作用或执行过程

三类进程：前台进程、后台进程、批处理进程；

后台进程（background process）是指在系统后台运行的进程。在一个系统中可能运行着很多一般用户不知道、也不关心的进程，如系统本身用于管理和控制的进程、各种服务器进程等。前台进程也可放在后台运行，但这时要用到I/O重定向。

▼ 试述0#和1#进程的作用及UNIX/Linux进程树的形成过程

在UNIX系统中0#进程是唯一只在核心态下执行的进程。它的功能有三点：调度分配处理机；负责进程交换；初始化时创建1#进程。在Linux系统中，0#进程在创建出1#进程后变成了空闲进程（Idler），当系统中没有其他进程就绪时它才运行，而它原来的功能被分散在几个内核进程中。

1#（init或systemd）进程是系统启动时创建的创建进程的进程。它的主要作用是根据启动配置文件的内容初始化系统、创建系统运行所需的进程。在Linux中，还要根据运行级别触发运行级别初始化事件，执行相应的脚本程序进行初始化。系统初始化完成后，1#进程变成了回收进程，专门领养没有父进程的孤儿进程或回收状态为ZOMBIE的僵尸进程。

从进程创建关系来看，0#进程创建了1#进程，它是1#进程的父进程。1#进程在系统启动过程中创建了系统所需要的其他进程，这些被创建的进程是1#进程的子进程，而子进程又可创建属于自己的子进程，因此系统中除了0#进程外，1#进程是其所有进程的祖先进程。

Linux系统的进程之间的家族关系可用命令pstree来查看。

▼ Linux系统中，进程常见状态标志有哪些？各是什么意思？

状态	说明
R	Running/Runnable，进程在执行中或在就绪队列排队
D	Delaying，非中断性睡眠，通常是因为等待I/O
S	Sleep，正在睡眠，等待事件发生
T	Traced or stopped，被跟踪或停止
Z	Zombie/Defunct，僵尸状态。进程已经终止，但父进程没有对它的结束做适当处理
W	Wait，无内存页面（被换出）（kernel 2.6.xx后不再使用）
N	低优先级进程
L	Locked，页面被锁进内存（用于实时控制或用户控制的I/O）
<	高优先级进程

▼ 登录shell是如何启动的？如何定制用户自己的shell启动

1.在系统启动shell过程中，依次进行以下操作。

- （1）执行系统profile文件/etc/profile（并在它的控制下执行/etc/profile.d/目录下的*.sh脚本文件）。
- （2）执行用户profile文件~/.bash_profile（Ubuntu用的是~/.profile）。
- （3）如果~/.bashrc存在则执行之。
- （4）如果/etc/bashrc文件存在则执行之。

需要说明的是，以上步骤都是在当前shell内执行的，目的是为用户设置环境变量或做相关准备。一切执行完毕之后出现系统提示符。

2.根据用户登录和shell的启动过程可以看出，用户可在shell的启动控制文件（/etc/profile，~/.bash_profile或~/.profile，/etc/bashrc和~/.bashrc）中加入自己的内容，而影响用户shell的启动过程，如设置或添加自己所需的环境变量等。如果要在这些文件中加入用户自己的内容，一般放在文件的最后，这样可以使用户在登录之后就进入某个特定程序，

当其退出时用户也随之注销。因此，可让用户只在某个规定封闭环境下工作，而不让其真正进入系统，看不到系统的提示符，从而可以提高系统的安全性和操作的方便性。

▼ 试述kill与killall的相同与不同点

kill用于向指定进程或进程组发送指定信号，如果未指定信号，则默认发送TERM（15）。若接收者收到信号，而没有对信号捕获并进行特殊处理，则它将被终止运行，也就是被杀掉了。

killall是另一个用来向指定进程发信号或终止进程的命令，它不像kill使用进程的PID作为操作对象，而是使用进程名。由于在系统中可能有多个同名进程同时运行，因此使用killall终止进程时可能会有扩大化的问题。

▼ 试描述suid、sgid和sticky机制，回答如何避免它们带来的安全隐患？分别查找系统中具有suid、sgid和sticky属性的文件。

对于可执行文件，有些文件还有另外两个与执行有关的属性。

（1）设置用户ID权限：setuid/suid。

（2）设置组ID权限：setgid/sgid。

#chmod u+s myp1 #为myp1设置suid。修改后的权限为rwsr-xr-x

#chmod g+s myp2 #为myp2设置sgid。修改后的权限为rwxrwsr-x

#chmod ug+s myp3 #为myp3同时设置suid和sgid。修改后的权限为rwsrwsrwx

#chmod 4755 myp1 #为myp1设置suid

#chmod 2775 myp2 #为myp2设置sgid

#chmod 6777 myp3 #为myp3同时设置suid和sgid

当一个程序具有suid属性时，它执行时的uid（有效uid）将是该程序所有者的uid，记为euid，而执行者原来的uid称为真实uid，记为ruid。此时，程序的执行者具有程序所有者的权限

当一个程序具有sgid属性时，它执行时的gid（有效gid）将是该程序的gid，记为egid，而执行者原来的gid称为真实gid，记为rgid。此时，程序的执行者具有程序所有者所在组的权限。

当一个属于超级用户的可执行二进制程序具有suid属性时，在一般用户执行它时，也就相当于有了超级用户的身份和权限。同样，当一个属于超级用户所在组的可执行二进制程序具有sgid属性时，当一般用户执行它时，也就相当于有了该组员的身份和权限。由此可见，suid和sgid权限的存在可能会给某些程序的执行带来方便，因为权限被放大了，但这也是一种危险，因为过大的权力可能会给系统的安全带来隐患，因此设置这种权限的可执行二进制程序不宜太多。

sticky权限只能运用于目录上，含义是该目录下所有的文件和子目录只能由所有者删除，即使其的权限是777或者其他。假如本来在该位上有x，则这些特别标志（suid，sgid，sticky）显示为小写字母（s，s，t）。否则，显示为大写字母（S，S，T）。

当一个目录具有sticky权限时，它内部的文件只能被它的所有者或超级用户删除、更名或移动。当一个可执行文件具有sticky权限时，在早期的系统中，程序执行时的正文段映像将被锁在交换区中，可使得程序装入更快速。

▼ 第七章 网络配置、管理及安全应用

▼ 一些命令

hostname：显示和临时设置主机名，其语法为：hostname [-v] {name|-F file}

hostnamectl：查询和设置主机名，其用法为：hostnamectl [options] {cmd}

ping命令的用法为：ping [options] hostname /ip

netstat -ia：显示所有TCP端口

ifconfig：管理网络接口，用于查看、配置、启用或禁用网络接口 功能是显示所有网卡的配置信息

IP地址(Internet Protocol Address)，又译为**网际协议地址格式**。每个IP地址都分为**网络号**和**主机号**两部分

由于一个IP地址由32位二进制数构成，将它分成4组，每组8位，各组都以十进制表示，并用圆点分开，这种表示方法称为“**点分十进制表示法**”

子网掩码可以用来区分一个IP地址的网络号和主机号 **各占多少位**。A、B、C类IP地址的标准子网掩码分别为：

A类—255.0.0.0

B类—255.255.0.0

C类—255.255.255.0

一个进程和另一个进程进行通信时，需要以某种方式标识自己。对此，TCP/IP用一个被称作**端口**的逻辑结构来实现。一个**端口号**是一个16位的数字，它唯一地标识一个进程。每个TCP或UDP分组头都包含源和目的端口号。

服务是**协议和端口**的组合。Linux作为一个网络操作系统最主要的功能就是提供。各种服务被定义在文件**/etc/services**中，此文件规定了**服务和端口与通信协议**的对应关系。

物理地址（MAC地址）一般位于网卡中，一个网卡具有全球**唯一的MAC**地址，用于标识网络设备，控制对网络介质的访问。

逻辑地址（网络地址）即IP地址或网络地址。IPv4使用32位二进制来表示网络地址，称为**IP地址**或逻辑地址。逻辑地址用于**网络层**上对目的主机的寻址。

netstat主要用于**显示系统网络信息**，包括**网络连接、路由表和网络接口通信统计信息**等。通过这些信息可以知道开启的端口、正在为哪些用户服务以及服务的状态等

显示网络接口使用的统计信息，使用**-i**选项执行**netstat**命令可以显示所有已配置接口的一些有用的统计信息，使用该参数，可以很容易检查连接的状态以及连接是否“正常”。因此，这是一个用于检测和排除网络故障的非常有用的措施。

第八章 shell程序设计

一些命令

chmod u+x ScripName：编写完后给脚本ScripName添加执行权限

文件状态测试有哪些？

-d	目录	-r	可读
-f	常规文件	-w	可写
-L	符号连接	-x	可执行
-s	文件长度大于0，非空	-u	文件有suid位设置

写出shell程序设计中的逻辑操作符及其功能

-a 逻辑与(and),操作符两边均为真，结果为真，否则为假。

-o 逻辑或(or),操作符两边一边为真，结果为真，否则为假。

! 逻辑否,条件为假，结果为真，反之。

写出shell程序设计中的字符串测试符号

=	两个字符串相等	-z	空串
!=	两个字符串不相等	-n	非空串

写出shell程序设计的数值测试符号

|--|--|--|

-eq	数值相等(equal to)	-lt	第一个数小于第二个数 (less than)
-ne	数值不相等(not equal to)	-le	第一个数小于等于第二个数
-gt	第一个数大于第二个数 (greater than)	-ge	第一个数大于等于第二个数

▼ 简答shell程序设计的选择结构

if语句: if then else 提供条件测试

▼ 简答shell程序设计的循环结构

case 语句：允许匹配模式、单词或值

for：每次依次处理列表内的信息，直至循环耗尽。

until:不常用。条件在循环末尾，至少执行一次。

while:条件在循环头部。

▼ shell编程



```
1 #!/bin/bash
2 groupadd class1
3 for i in {9901..9930}
4 do
5 xx=$(echo $i | sed 's/99//');
6 useradd -g class1 std$xx -p ""
7 done
```

加入一个新组为class1，然后加入属于这个组的30个用户，username的形式为stdxx，当中xx从01到30



```
1 #!/bin/bash
2 for file in $(ls $2)
3 do
4 bname=$(grep -l $1 $2/$file)
5 basename $bname
6 done
```

在某个目录下查找含有指定字符串的文件



```
1 #!/bin/sh
2 factorial=1
3 for a in `seq 1 10`
4 do
5 factorial=`expr $factorial \* $a `
6 done
7 echo "10! = $factorial"
```

循环计算10的阶乘



```

1 #!/bin/bash
2 for filename in `ls`
3 do
4 if test -d $filename
5 then b=0
6 else
7 a=$(ls -l $filename | awk '{ print $5 }')
8 if test $a -eq 0
9 then rm $filename
10 fi
11 fi
12 done

```

删除当前文件夹下大小为0的文件

编写脚本查看你当前目录下的文件属性（是普通文件还是目录）。考察的是for语句和判断语句if...else...

```

▼
1 #!/bin/bash
2
3 for i in *
4 do
5     if [ -f $i ]
6     then
7         echo "$i is a file"
8     else
9         if [ -d $i ]
10        then
11            echo "$i is a directory"
12        fi
13    fi
14 done

```

写出下列程序的输出结果

Input username:

qiang

usr qiang online

Input username:

xiao

usr xiao offlineInput username:

```

▼
1 #!/bin/bash
2 check_user()
3 {
4     usr=`who | grep $1 | wc -l`
5     if [ $usr -eq 0 ]
6     then
7         return 0
8     else
9         return 1
10    fi
11 }
12
13 while true
14 do

```

```
15     echo "Input username:"
16     read username
17     check_user $username
18     if [ $? -eq 1 ]
19     then
20         echo "usr $username online"
21     else
22         echo "usr $username offline"
23     fi
24 done
```

利用vi编辑器编辑脚本，将脚本命名为：**脚本功能.sh**

Shell脚本不是复杂的程序，它是按行解释的，脚本第一行总是以`#!/bin/sh`开头，它通知系统以下的Shell程序使用系统上的Bourne Shell来解释。

习惯上，第二行**注释中**写入脚本名，第三行注释开始写入**脚本功能**

`/dev/null` 是一个虚设的设备，俗称**Linux黑洞**,任何对`/dev/null`的写入都会**成功**，但数据会**消失得无影无踪**，没有任何反馈。所以经常把不想在屏幕显示的信息全部送到`/dev/null`中，在shell脚本中用得比较多。如：`ls -l > /dev/null`

变量可以定制用户本身的工作**环境**，使用变量可以**保存有用**信息，使**系统**获知用户相关设置，也用于保存暂时信息。

使用**echo命令**可以显示单个变量的取值，要在变量名前面加**\$**

如果设置变量时，不想再改变其值，可以将其设置为 **只读**，只要在变量名前加**readonly**。

使用**set命令**可以显示所有本地定义的shell变量。使用 **unset命令**可以清除对变量的定义。

本地变量是用户自定义的变量。**环境变量**是用于所有用户变量，用于用户进程前，必须用**export命令**导出。

位置变量：**\$0**是脚本名，**\$1-\$9**是脚本参数。

运行脚本ScripName的方法是：**ScripName**